DGA Classifier from Domain Names: Family-First or Direct Binary?

Roudy Barakat rtb07@mail.aub.edu
Abdo Sebailli ams86@mail.aub.edu
American University of Beirut

1 Problem and Motivation

Malware uses Domain Generation Algorithms (DGAs) to create many domains from a shared seed, so infected hosts can reach the attacker without a fixed address [12]. We detect DGAs from the domain name alone. This is hard because families differ: some produce random strings (`gqspwpsurdun.ru`, CryptoLocker) or hex hashes (Bamital), while others join dictionary words (`kneemanualgirl.com`) and look legitimate.

Question. For a DGA/legitimate decision, is it better to train on the DGA families and collapse the prediction (*family-first*), or to train on DGA/legitimate directly (*binary*)?

Why it matters. Name-based filters, e.g. in DNS resolvers, must judge a query before any connection is made. If family-first wins, family labels are worth collecting even when only a binary decision is needed; if not, binary labels are enough.

Goal. For each model, the difference in macro-F1 between family-first and binary on the same test set, over 5 seeds. Family-first counts as better if its mean is higher by more than the standard deviation.

2 Impact

If it succeeds. Teams building name-based DGA filters learn whether family labels are worth collecting. Per-family results show which DGAs (random-string or dictionary-based) are missed.

If it fails. Families that neither setup separates from legitimate domains, likely dictionary-based ones, need network features such as the NXDOMAIN rate or query periodicity [1]. Knowing which families these are is itself useful.

Risks.

- *Unequal errors:* a false negative leaves a host talking to its attacker; a false positive blocks a real service. Since most DNS traffic is legitimate, even a small false-positive rate means many alarms.
- *Data bias:* Tranco [8] favours popular sites, so unusual legitimate names (non-English, short, CDN hosts) may be flagged. Families with fewer than 300 domains are merged into one class, so we report no per-family results for them.
- *Validity:* we test only families seen in training; new or adversarial DGAs may not be detected.

3 Related Work

Woodbridge et al. [16] trained a character-level LSTM on 30 DGA families: a binary model reached an AUC of 0.9993, but a separate family model only 0.54 macro-F1 (0.96 when similar families were grouped). They never collapsed the family model into a detector, which is the comparison we make.

Gregório et al. [6] classified 61 families with a character-level CNN (average F1 84%, some families at 0%), but with no legitimate class. We add that class and test whether family confusions still matter after collapsing.

Torrealba Aravena et al. [14] (DeepDGA, BiLSTM with word embeddings) exceed 0.97 precision on dictionary DGAs, where feature-based models stay below 0.89; they use a bigger architecture, we test richer labels.

Shahzad et al. [13] found little difference between LSTM, BiLSTM and GRU (AUC $\approx$ 0.98) for binary detection. Since the architecture barely mattered, we change the labels instead; we also follow their 80/10/10 split.

Dahal and Kim [3] reached 88% accuracy with autoencoded character counts, falling to 78% on unseen families; we test known families only, so this drop marks the limit of our results.

4 Data

Sources. DGA domains come from Gregório and Cansian (Mendeley, 2025, CC BY 4.0 [5]; 3,090,661 domains, 160 families, from DGArchive [12]) and Givre (Kaggle, 2021, license not stated, academic use [4]; 80,000 domains, 7 families). Legitimate domains come from the Tranco top 1M [8] (list `64VVX`, `https://tranco-list.eu/list/64VVX`); the DGA datasets' legitimate rows are not used.

Size, input and target. 1,999,335 domains: 999,664 legitimate and 999,671 DGA in 103 families (Fig. 1). The input is the full domain name, TLD included; all features are derived from it. The targets are the binary label and the family (`legit` or one of 103). DGA domains are longer (mean 18.4 vs. 13.7 characters), have higher character entropy (3.17 vs. 2.79 bits) and more digits (15% vs. 4%). We use pandas [9] and Matplotlib [7].

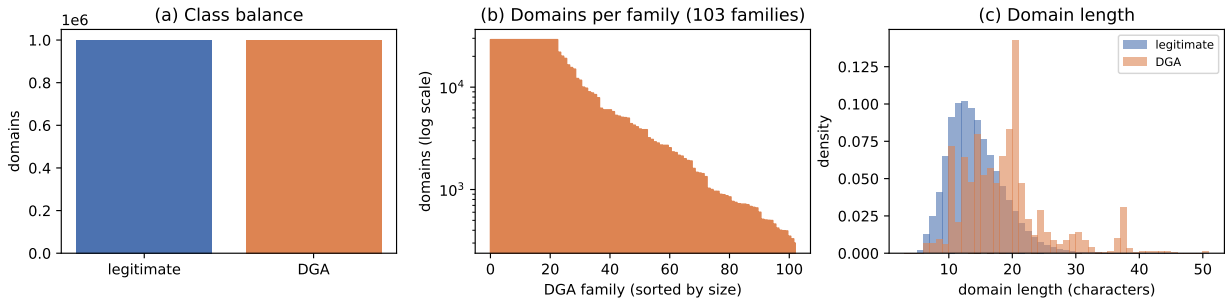


Figure 1: (a) Class sizes; (b) domains per DGA family (log scale); (c) domain length.

Cleaning. No missing values. We drop 330 domains found in both Tranco and a DGA list (e.g. `pywolwnvd.biz`), 141 duplicates and 7 invalid strings, and map Kaggle family names to Mendeley's.

Balancing. The 161 cleaned families range from 2 to 519,280 domains. We keep all legitimate domains, merge the 59 families under 300 domains into `other_dga`, and cap each family at 29,120

so that DGA and legitimate are equal in number. Sample weights give legitimate and DGA half each and, within DGA, are proportional to $1/\sqrt{n_f}$ (n_f: family size; maximum ratio 9.9).

Biases. Tranco favours popular sites. 23 DGA families use a single TLD (legitimate domains use 1,030), so the TLD alone separates some families. DGA data cover only 2021 and late 2025.

5 Proposed Models and Methods

Every model is trained in two setups on the same data: *binary* (DGA/legitimate) and *family-first* (`legit` or one of 103 families, collapsed with $P(\text{DGA}) = 1 - P(\texttt{legit})$). Neural models are also trained *multi-task* [2]: a shared encoder with a binary and a family head whose losses are summed, so family labels shape the representation without a collapse step.

Model 0: majority class. The floor every model must beat.

Model 1: logistic regression [11]. Fast and interpretable, but linear: in the binary setup it draws one boundary around very different families, while family-first gives one per family. If family labels help anywhere, it should be here.

Model 2: character-level CNN [6] (PyTorch [10]). Convolutions learn local patterns (hex runs, vowel–consonant alternation, word pieces), which is where families differ. It shows whether the gain from family labels shrinks as the model gets more flexible.

Model 3: character-level Transformer [15]. Self-attention relates characters across the whole name, which suits dictionary DGAs, the hardest case [14], built from whole words. We also test whether CNN or Transformer embeddings fed to Model 1 match the full neural models.

6 Evaluation

Metrics (on the DGA/legitimate decision unless stated). *Macro-F1* is the main metric: it averages both classes' F1, so favouring one class does not pay, and Woodbridge et al. [16] use it. *DGA precision and recall* capture the unequal error costs (Section 2). *ROC-AUC* is threshold-free and allows comparison with published work. *Recall per family* shows which families, e.g. dictionary-based ones, are missed. Accuracy alone is not used, as it hides which errors are made.

Data split. One fixed 80/10/10 train/validation/test split, stratified by family and shared by all models and setups. Hyperparameters are tuned on validation; the test set is used once. Results are mean ± standard deviation over 5 seeds.

Comparisons. *Baseline:* Model 0 is the floor, and each model's binary setup is the reference for family-first and multi-task. *Ablations:* the three setups per model, and each model with and without the TLD, since the TLD alone separates some families. *Published results:* AUC of 0.9993 [16] and ≈0.98 [13], family F1 of 84% [6]; on other data, so only indicative.

7 Resources and Code

Code: `https://github.com/RoudyBarakat/dga-detection`, with the data loading and balancing scripts, the baselines (Models 0 and 1) and a README with setup instructions. Raw datasets are not redistributed; the README links to them.

8 Timeline

The dataset is ready (Section 4).

Week	Tasks	Milestone
Oct 19–25	Models 0–1 on full data, both setups, with/without TLD; CNN code on the shared loader	Baselines done
Oct 26–Nov 1	CNN binary and family-first; tune Model 1, 5 seeds	CNN trained
Nov 2–8	Multi-task CNN; Transformer code	
Nov 9–15	Transformer, three setups; CNN 5 seeds and TLD ablation	All models trained
Nov 16–22	Final test runs, per-family analysis, embeddings fed to Model 1	Results frozen
Nov 23–29	Report writing	Report draft
Nov 30–Dec 6	Slides, rehearsal, final presentation	Presentation

References

[1] Manos Antonakakis, Roberto Perdisci, Yacin Nadji, Nikolaos Vasiloglou, Saeed Abu-Nimeh, Wenke Lee, and David Dagon. From throw-away traffic to bots: Detecting the rise of DGA-based malware. In *Proceedings of the 21st USENIX Security Symposium*, pages 491–506, 2012.

[2] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997. doi: 10.1023/A:1007379606734.

[3] Binay Dahal and Yoohwan Kim. Autoencoded domains with mean activation for DGA botnet detection. In *Proceedings of the 12th IEEE International Conference on Global Security, Safety and Sustainability (ICGS3)*. IEEE, 2019.

[4] Charles Givre. DGA dataset. `https://www.kaggle.com/datasets/gtkcyber/dga-dataset`, 2021. Version 1; accessed October 2026.

[5] João Rafael Gregório and Adriano Mauro Cansian. Legitimate domains and DGA categorized morphologically and by families, 2025. URL `https://data.mendeley.com/datasets/nhvyvytn2h/1`.

[6] João Rafael Gregório, Adriano Mauro Cansian, and Leandro Alves Neves. Class incremental deep learning: A computational scheme to avoid catastrophic forgetting in domain generation algorithm multiclass classification. *Applied Sciences*, 14(16):7244, 2024. doi: 10.3390/app14167244.

[7] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

[8] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. Tranco: A research-oriented top sites ranking hardened against manipulation. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019. doi: 10.14722/ndss.2019.23386.

[9] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

[10] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas

Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.

[11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] Daniel Plohmann, Khaled Yakdan, Michael Klatt, Johannes Bader, and Elmar Gerhards-Padilla. A comprehensive measurement study of domain generating malware. In *Proceedings of the 25th USENIX Security Symposium*, pages 263–278, 2016.

[13] Haleh Shahzad, Abdul Rahman Sattar, and Janahan Skandaraniyam. DGA domain detection using deep learning. In *Proceedings of the 5th IEEE International Conference on Cryptography, Security and Privacy (CSP)*. IEEE, 2021. doi: 10.1109/CSP51677.2021.9357591.

[14] Lucas Torrealba Aravena, Pedro Casas, Diego García, Javier Bustos-Jiménez, and Ivana Bachmann. Malicious domain names detection with DeepDGA, a hybrid character and word embeddings deep learning architecture. In *Proceedings of the 21st International Conference on Network and Service Management (CNSM)*. IFIP, 2025. doi: 10.23919/CNSM67658.2025.11297537.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[16] Jonathan Woodbridge, Hyrum S. Anderson, Anjum Ahuja, and Daniel Grant. Predicting domain generation algorithms with long short-term memory networks, 2016.